

PROYECTO NOVAPAY

GRUPO 3

Sistema de Detección de Fraude Financiero

Integrantes Grupo 3

Fernando Hidalgo

Alejandro Lucas

Asier Varela

Marco José Guevara

Rafael Mattos

Antonio Soler

Ariela Astorga

Manuel Correa

Daniel Lavía

José Marín

Jaime Vázquez

ÍNDICE

1. Resumen ejecutivo
2. Introducción
3. Objetivo del proyecto
4. Metodología
5. Análisis exploratorio de datos
 - 5.1 Descripción del dataset
 - 5.1.1 Origen de los datos
 - 5.1.2 Descripción del dataset y sus variables
 - 5.1.3 Descripción de la variable objetivo
 - 5.2 Preparación y limpieza de datos
 - 5.2.1 Distribución de clases y desbalanceo
 - 5.2.2 Calidad de datos
 - 5.2.3 Valores nulos
 - 5.2.4 Duplicados
 - 5.2.5 Valores inconsistentes y formatos
 - 5.2.6 Outliers
 - 5.2.7 Variables con poca variabilidad
 - 5.2.8 Variables descartadas para el modelo
 - 5.3 Análisis de la variable objetivo
 - 5.3.1 Proporción de fraude vs no fraude
 - 5.3.2 Consecuencias del desbalance para el modelado
 - 5.4 Análisis univariante
 - 5.4.1 Variables numéricas
 - 5.4.2 Variables categóricas
 - 5.4.3 Variables binarias
 - 5.5 Análisis bivalente
 - 5.6 Análisis temporal
 - 5.7 Detección de patrones de fraude
 - 5.8 Ingeniería de variables
 - 5.9 Modelado
 - 5.10 Evaluación de resultados
6. Estructura de la base de datos
 - 6.1 Modelo de Entidad Relación
 - 6.2 Modelo lógico de la base de datos y normalización
7. Arquitectura del proyecto
 - 7.1 Descripción de flujo de trabajo y tecnologías utilizadas

- 7.2 Kafka
 - 7.2.1 Generación de datos - Producer
 - 7.2.2 Consumer
- 7.3 SHAP
- 7.4 Portal del analista - Full Stack
- 7.5 Detalles técnicos Ciberseguridad
- 8. Sistema de reglas de negocio
 - 8.1 Descripción general
 - 8.2 Flujo de decisión
 - 8.3 Detección de señales
 - 8.4 Clasificación de alertas
 - 8.5 Detección del tipo de fraude
 - 8.6 Acción recomendada
 - 8.7 Relación con el modelo ML
- 9. Conclusiones
- 10. Herramientas y tecnologías
- 11. Limitaciones y líneas futuras

Resumen ejecutivo

NovaPay es un sistema de detección de fraude financiero en tiempo real desarrollado sobre un dataset sintético de 100.000 transacciones. El modelo XGBoost entrenado obtiene un ROC-AUC de 0.99 y un recall de 0.79 sobre la clase fraudulenta, detectando automáticamente casi ocho de cada diez fraudes. El sistema integra una API en producción desarrollada con FastAPI, persistencia en PostgreSQL, enriquecimiento en tiempo real con Valkey, ingesta de eventos mediante Apache Kafka y explicabilidad de predicciones con SHAP. Cada transacción evaluada, genera una lista de razones comprensibles para el analista, que puede confirmar o descartar la alerta a través del portal de revisión. El proyecto ha sido desarrollado por un equipo multidisciplinar en tres verticales: datos y machine learning, plataforma e infraestructura, y ciberseguridad.

2. Introducción

El presente proyecto se centra en el desarrollo de un sistema de detección de fraude a partir de un conjunto de datos transaccionales, en el marco de un desafío orientado al análisis de datos y al aprendizaje automático aplicado a un caso realista de riesgo operacional.

El problema abordado consiste en identificar, entre un elevado volumen de operaciones, aquellas transacciones que presentan indicios de comportamiento fraudulento, con el fin de reducir pérdidas económicas, mejorar los procesos de supervisión y apoyar la toma de decisiones en entornos donde la rapidez y la precisión son especialmente relevantes.

La detección de fraude constituye un problema de gran interés dentro de la analítica avanzada, ya que combina varios retos habituales en ciencia de datos: presencia de variables heterogéneas, necesidad de interpretar patrones de comportamiento, posible desbalance entre clases y dificultad para diferenciar operaciones legítimas de aquellas que imitan comportamientos normales.

En este contexto, el proyecto no solo requiere construir un modelo predictivo, sino también comprender en profundidad la estructura de los datos, evaluar su calidad, extraer variables informativas y justificar las decisiones metodológicas adoptadas a lo largo del proceso analítico.

Desde una perspectiva aplicada, este trabajo pretende simular un flujo completo de resolución de un problema de fraude: comprensión del negocio, análisis exploratorio, preparación de los datos, construcción de modelos, evaluación de resultados e interpretación de hallazgos.

De esta manera, la memoria documenta no sólo el resultado final obtenido, sino también el razonamiento técnico seguido para transformar un conjunto de datos brutos en una solución analítica estructurada, reproducible y útil para un posible escenario de decisión.

3. Objetivo del proyecto

El objetivo general del proyecto es desarrollar un sistema integral de detección de fraude financiero en tiempo real, capaz de identificar transacciones potencialmente fraudulentas a partir del análisis de variables transaccionales, temporales y de comportamiento, siguiendo una metodología completa y documentada que abarca desde la analítica de datos hasta el despliegue en producción.

Este objetivo se articula en torno a tres verticales complementarias. El equipo de datos y machine learning asume la responsabilidad de construir un modelo predictivo robusto, maximizando la capacidad de detección de operaciones sospechosas mediante técnicas de clasificación supervisada, ingeniería de variables y gestión del desbalanceo de clases. El equipo de plataforma e infraestructura desarrolla la arquitectura tecnológica que sostiene el sistema en producción: la API de predicción, las bases de datos de persistencia y enriquecimiento en tiempo real, la ingesta de eventos mediante Kafka y el despliegue dockerizado del conjunto de servicios. El equipo de ciberseguridad incorpora una capa de auditoría y control orientada a supervisar el comportamiento del sistema, reforzar los mecanismos de revisión de operaciones sospechosas y garantizar que la plataforma opera con las salvaguardas necesarias en un entorno fintech.

El resultado esperado no es únicamente un modelo predictivo, sino una plataforma integrada en la que cada componente cumple una función específica dentro de un flujo coherente, auditable y defendible: desde la llegada de una transacción hasta la decisión del analista que la revisa en el portal.

4. Metodología

La metodología del proyecto se estructura en varias fases consecutivas, siguiendo un enfoque estándar de ciencia de datos orientado a problemas de clasificación supervisada.

En primer lugar, se realiza una comprensión inicial del problema y del dataset que se requiere para poder generar un modelo de análisis. Se identifica la variable objetivo, la naturaleza de cada columna y el posible papel de cada atributo dentro de la detección del fraude.

En segundo lugar, se lleva a cabo un análisis exploratorio de datos con el propósito de estudiar distribuciones, relaciones entre variables y posibles patrones asociados al fraude. En esta fase se revisan estadísticas descriptivas, frecuencias de variables categóricas, comportamiento de la variable objetivo, correlaciones entre variables numéricas y diferencias entre transacciones fraudulentas y no fraudulentas. También se presta atención a elementos que puedan afectar a la calidad del análisis, como registros atípicos, formatos incorrectos, sesgos o valores extremos.

La tercera fase corresponde a la creación de la base de datos con datos sintéticos y preparación del conjunto de datos para su uso en modelos de machine learning. Cuando sea necesario, se podrán generar nuevas variables derivadas que ayuden a representar mejor el comportamiento del usuario o el contexto de la transacción.

Posteriormente, se aborda la fase de modelado, en la que se entrenan distintos algoritmos de clasificación para predecir la variable `es_fraude`. Se comparan los resultados de distintos modelos, prestando especial atención a indicadores más informativos en contextos de fraude, como la precisión y el recall.

Se diseñó y levantó una base de datos PostgreSQL, mientras que PGAdmin se empleó como herramienta de administración, consulta y visualización de la base de datos.

Se desarrollaron las APIs necesarias mediante FastAPI, creando los endpoints requeridos para la integración con la parte Full Stack, y se implementó una arquitectura de ingesta basada en Kafka para insertar flujos de transacciones en tiempo casi real, aprovechando su modelo de publicación y lectura de eventos para desacoplar productores y consumidores dentro del sistema.

El flujo completo de ingeniería de datos, desarrollo de plataforma y analítica de fraude, fue desplegado sobre contenedores Docker, con el objetivo de garantizar reproducibilidad, modularidad y facilidad de despliegue.

El equipo Full Stack trabajó como capa de integración entre los modelos y datos proporcionados por Data Science, las necesidades de seguridad definidas para el acceso al sistema y la experiencia final del analista.

La metodología se basó en dividir el desarrollo en dos bloques principales: backend y frontend. En el backend se definió una API propia para centralizar la autenticación, proteger las rutas privadas y exponer al frontend los datos necesarios para la revisión de transacciones y clientes. Esta API también funciona como intermediaria frente a los servicios externos de Data Science.

En el frontend se construyó progresivamente el portal operativo, priorizando primero las funcionalidades principales y después la experiencia visual, la adaptación responsive y la claridad de uso. El desarrollo se organizó en iteraciones cortas, validando cada bloque funcional antes de pasar al siguiente: autenticación, dashboard, transacciones, detalle de operación, clientes y acciones de revisión.

La colaboración con Data Science permitió consumir información transaccional y resultados del modelo de detección de fraude, mientras que la colaboración con Ciberseguridad ayudó a orientar el control de acceso y la protección de las rutas internas. De esta forma, Full

Stack actuó como puente entre los datos, la seguridad y el producto final utilizado por el analista.

Por último, la vertical de ciberseguridad asumió el rol de Red Team dentro del esquema adversarial del proyecto, encargándose de generar el comportamiento fraudulento contra el que se evalúa el modelo.

Su metodología se estructuró en dos rondas iterativas: una primera ronda de línea base, orientada a caracterizar el comportamiento del clasificador frente a un catálogo de tipologías de fraude (suplantación de identidad, pruebas de tarjeta, casos límite y combinaciones de variables de alto riesgo), y una segunda ronda de adaptación, ejecutada sobre el modelo ya reentrenado, en la que el adversario ajusta sus tácticas a partir de los puntos ciegos detectados en la primera.

Todas las pruebas se realizaron en modo gray-box —con conocimiento del esquema de la API, del conjunto de entrenamiento y de la importancia de variables del modelo, pero sin acceso a su implementación interna— mediante un cliente Python reproducible (semilla fija, limitación de frecuencia y registro versionado en CSV) que garantiza la trazabilidad de cada petición.

De forma complementaria, se llevó a cabo un análisis de superficie de ataque y una auditoría de seguridad de la plataforma siguiendo el enfoque del SSDLC y el OWASP Top 10. El detalle de tipologías, métricas de evasión y hallazgos de auditoría se documenta en los informes específicos de Red Team de cada ronda.

5. Análisis exploratorio de datos

5.1 Descripción del dataset

El dataset de trabajo está formado por 100.000 registros y 16 variables, incluyendo identificadores de transacción y de usuario, variables temporales, variables numéricas asociadas al comportamiento de compra y variables categóricas y binarias relacionadas con el contexto de la operación, como el país de emisión, la categoría comercial, el tipo de tarjeta, el uso de VPN o la verificación mediante 3D Secure. La variable objetivo del problema es **es_fraude**, una etiqueta binaria que permite formular el proyecto como una tarea de clasificación supervisada, en la que el propósito final es discriminar transacciones legítimas de transacciones fraudulentas a partir de patrones observables en los datos.

5.1.1 Origen de los datos

Primero se hicieron pruebas con un dataset de la página de Kaggle, este es el nombre del archivo: **dataset_fraude_riesgo_pais_ult**.

La idea era investigar qué campos son los que se tienen en un dataset real, e identificar cuáles son útiles para el análisis.

Posteriormente los datos fueron generados de forma sintética. La base de datos de transacciones se generó con ayuda de un LLM, y guiando ciertas condiciones, de manera que siguiera una estructura balanceada de transacciones fraudulentas. Por ejemplo, que el 12% estuviera etiquetando como fraude o que el 60% de los fraudes son con el uso de VPN.

La base de datos de clientes se generó en Python usando Faker.

4.1.2 Descripción del dataset y sus variables

El dataset contiene 100.000 registros de transacciones. Se trabajó con 16 columnas. Las primeras dos variables `id_transaccion` e `id_usuario` (str) se crearon de tipo UUID para mayor seguridad en caso de ataques a la BD.

A continuación se muestra un detalle de las columnas, su tipo y descripción.

Columna	Tipo	Descripción
<code>id_transaccion</code>	Categórica / str (UUID)	Identificador único de cada transacción, sirve para trazabilidad. Ejemplo: <code>e75696a8-e629-47d4-a8f8-90f0990b8ac7</code>
<code>id_usuario</code>	Categórica / str (UUID)	Identificador único del usuario que realiza la transacción.
<code>fecha</code>	Temporal / str	Fecha en la que se realiza la transacción. Permite estudiar patrones temporales. Ejemplo: <code>2025-01-01 00:05:00</code>
<code>hora</code>	Numérica discreta / int64	Hora de la transacción. Los valores van entre 0 y 24.
<code>dias_antiguedad_cuenta</code>	Numérica discreta / int64	Número de días desde la creación de la cuenta. Ejemplo: 1268.

Columna	Tipo	Descripción
email_verificado	Binaria / int64	Indica si el email del usuario está verificado (1) o no (0).
pais_emision	Categórica / str	País emisor de la tarjeta o del medio de pago. Ejemplo: RU.
categoria	Categórica / str	Categoría de gasto o comercio, por ejemplo, supermercado, viajes, etc.
importe	Numérica continua / float64	Importe de la transacción. Muy relevante para detectar patrones anómalos.
es_online	Binaria / int64	Indica si la compra es online (1) o presencial (0).
pais_pago	Categórica / str	País donde se realiza el pago. Permite comparar con el país de emisión.
tipo_tarjeta	Categórica / str	Tipo de tarjeta utilizada, por ejemplo: crédito o débito.
mismo_envio_facturacion	Binaria / int64	Indica si la dirección de envío coincide con la de facturación (1) o no (0).
tipo_dispositivo	Categórica / str	Tipo de dispositivo usado en la operación. 4 valores: Móvil, Tablet, Desktop, Físico (TPV).
uso_vpn_proxy	Binaria / int64	Indica si se detecta uso de VPN o proxy (1) o no (0).
paso_3d_secure	Binaria / int64	Indica si la transacción pasó la autenticación 3D Secure (1) o no (0).
es_fraude	Binaria / objetivo / int64	Variable objetivo. Indica si la transacción es fraudulenta (1) o legítima (0).

5.1.3 Descripción de la variable objetivo

La variable objetivo del problema es **es_fraude**, una variable binaria de tipo entero que toma el valor 1 cuando la transacción ha sido etiquetada como fraudulenta y el valor 0 cuando se considera legítima. Su presencia en el dataset permite formular el problema como una tarea de clasificación supervisada binaria, en la que el modelo debe aprender a distinguir, a partir del resto de variables, qué transacciones presentan un patrón compatible con el fraude.

Desde el punto de vista de negocio, esta variable representa la señal más valiosa del dataset: cada registro etiquetado como fraude es un caso confirmado de comportamiento anómalo que el sistema debe ser capaz de anticipar antes de que se produzca o, al menos, de detectar en tiempo real para minimizar el impacto económico.

5.2 Preparación y limpieza de datos

5.2.1 Distribución de clases y desbalanceo

El análisis de la distribución de la variable objetivo revela un desbalance pronunciado entre clases. Del total de 100.000 registros, aproximadamente el 2% corresponde a transacciones fraudulentas y el 98% restante a transacciones legítimas. Esta proporción es representativa de escenarios reales de fraude financiero, donde los casos positivos son necesariamente minoritarios respecto al volumen total de operaciones.

Este desbalance tiene implicaciones directas sobre el proceso de modelado. Un clasificador que prediga siempre la clase mayoritaria obtendría una exactitud (accuracy) del 98% sin detectar ningún caso de fraude, lo que evidencia que esta métrica no es adecuada para evaluar el rendimiento del modelo en este contexto. En su lugar, se priorizan métricas que reflejan la capacidad real de detección, especialmente el recall sobre la clase positiva, que mide la proporción de fraudes reales que el sistema es capaz de identificar.

La decisión de maximizar el recall frente a la precisión responde a una valoración del coste asimétrico de los errores: un falso negativo —fraude no detectado— implica una pérdida económica directa y un riesgo reputacional para la entidad, mientras que un falso positivo —transacción legítima bloqueada— supone una fricción para el usuario que, aunque indeseable, es recuperable. En consecuencia, el sistema acepta un número controlado de falsas alarmas a cambio de reducir al mínimo los fraudes que pasan desapercibidos.

Para compensar el desbalance durante el entrenamiento, se aplica la técnica SMOTE (Synthetic Minority Oversampling Technique) exclusivamente sobre el conjunto de entrenamiento, generando muestras sintéticas de la clase minoritaria sin alterar la distribución real del conjunto de validación ni del de test.

5.2.2 Calidad de datos

Antes de abordar el análisis exploratorio y el modelado, se realizó una revisión sistemática de la calidad del dataset con el objetivo de identificar problemas que pudieran comprometer la validez del análisis posterior. Esta revisión abarcó la detección de valores nulos, registros duplicados, inconsistencias en la codificación de variables y comportamiento de valores extremos.

5.2.3 Valores nulos

El dataset no presenta valores nulos en la mayoría de sus columnas. La única excepción relevante es la variable `tipo_ip`, que registra aproximadamente 39.800 valores ausentes sobre el total de 100.000 registros. Tras analizar el origen de esta ausencia, se concluye que no se trata de un error de recogida de datos sino de un comportamiento esperado: los registros sin valor en `tipo_ip` corresponden a transacciones realizadas de forma presencial a través de datáfono (TPV), operaciones que por su naturaleza no generan dirección IP. En consecuencia, estos valores nulos se imputan con la categoría `Sin_IP`, preservando la información sobre el canal de la transacción sin introducir sesgos artificiales en el modelo.

5.2.4 Duplicados

No se detectaron registros duplicados en el dataset. Cada fila corresponde a una transacción única identificada mediante un `id_transaccion` de tipo UUID, lo que garantiza la trazabilidad individual de cada operación y elimina el riesgo de que un mismo evento sea contabilizado más de una vez durante el entrenamiento.

5.2.5 Valores inconsistentes y formatos

Las variables categóricas presentan valores consistentes y sin variaciones de formato que requieran corrección. Los códigos de país siguen el estándar ISO 3166-1 alpha-2 en todos

los registros. Las variables binarias (email_verificado, es_online, mismo_envio_facturacion, uso_vpn_proxy, paso_3d_secure, es_fraude) toman exclusivamente los valores 0 y 1, sin presencia de valores intermedios ni codificaciones alternativas.

La variable fecha se almacena como cadena de texto en formato YYYY-MM-DD HH:MM:SS y requiere conversión explícita a tipo datetime para poder extraer las variables temporales derivadas que se describen en la sección de ingeniería de variables.

5.2.6 Outliers

La variable importe presenta una distribución asimétrica con valores extremos en la cola derecha, lo cual es coherente con el comportamiento real de las transacciones financieras, donde la gran mayoría de operaciones corresponden a importes bajos o medios y una minoría corresponde a compras de alto valor. Dado que estos valores extremos no son errores sino observaciones legítimas con potencial señal informativa para la detección de fraude, se opta por no eliminarlos ni transformarlos, y se delega en el modelo la tarea de aprender su relevancia relativa.

5.2.7 Variables con poca variabilidad

Se revisó la varianza de todas las variables numéricas y la distribución de frecuencias de las categóricas. No se identificaron variables con varianza prácticamente nula ni categorías dominantes que pudieran inutilizar una variable como predictor. La variable uso_vpn_proxy presenta una proporción muy baja de valores positivos (aproximadamente el 1% de los registros), lo que refleja la realidad del fenómeno pero no invalida su uso como feature, dado que su capacidad discriminatoria frente a la clase fraude es estadísticamente relevante.

5.2.8 Variables descartadas para el modelo

Las columnas id_transaccion, id_usuario y fecha se excluyen del conjunto de features del modelo. Los identificadores son variables de trazabilidad sin capacidad predictiva directa, y la fecha es sustituida por las variables temporales derivadas que se describen en la sección de ingeniería de variables. Esta decisión evita que el modelo aprenda patrones espurios asociados a identificadores específicos que no se generalizarán a datos nuevos.

5.3 Análisis de la variable objetivo

5.3.1 Proporción de fraude vs no fraude

El análisis de la distribución de es_fraude confirma el fuerte desbalance entre clases descrito en la sección anterior. Del total de 100.000 transacciones, aproximadamente 2.000 corresponden a casos de fraude (2%) y 98.000 a transacciones legítimas (98%). Esta proporción es característica de los problemas de detección de fraude en entornos financieros reales, donde la clase positiva es estructuralmente minoritaria por razones inherentes al propio fenómeno.

5.3.2 Consecuencias del desbalance para el modelado

El desbalance extremo entre clases introduce varios riesgos metodológicos que deben abordarse de forma explícita durante el proceso de modelado.

En primer lugar, los algoritmos de clasificación estándar tienden a optimizar métricas globales como la exactitud, lo que los lleva a favorecer la clase mayoritaria en detrimento de la minoría. Un modelo que clasifique todas las transacciones como legítimas obtendría una exactitud del 98% sin detectar ningún caso de fraude, resultado que es formalmente bueno pero operativamente inútil.

En segundo lugar, el desbalance puede hacer que el modelo asigne probabilidades sistemáticamente bajas a la clase positiva, dificultando la calibración del umbral de decisión. Por este motivo, se presta especial atención a la curva precision-recall y se analiza el comportamiento del modelo a distintos umbrales, seleccionando aquel que maximiza el recall sin degradar la precisión por debajo de un nivel aceptable para el negocio.

En tercer lugar, la evaluación del modelo requiere métricas adaptadas al contexto. La exactitud queda descartada como métrica principal. En su lugar se utilizan el recall, la precisión, el F1-score y el área bajo la curva ROC (ROC-AUC), que proporcionan una visión más completa del comportamiento del clasificador frente a ambas clases.

Métricas recomendadas para este tipo de problema

Dada la naturaleza del problema y el coste asimétrico de los errores, las métricas se priorizan en el siguiente orden:

- **Recall (sensibilidad):** mide la proporción de fraudes reales que el modelo detecta correctamente. Es la métrica principal del proyecto, dado que un fraude no detectado tiene consecuencias económicas directas y potencialmente irreversibles.
- **Precisión:** mide la proporción de alertas generadas que corresponden a fraudes reales. Una precisión muy baja implica un exceso de falsos positivos que puede saturar al equipo de análisis y generar fricción innecesaria con los usuarios.
- **F1-score:** media armónica entre precisión y recall, útil para evaluar el equilibrio entre ambas métricas en un único indicador.
- **ROC-AUC:** mide la capacidad discriminativa del modelo de forma independiente al umbral de decisión. Un valor cercano a 1 indica que el modelo separa bien ambas clases en términos de probabilidad.

El modelo final obtiene un recall de 0.94 sobre la clase fraude, una precisión de 0.77, un F1-score de 0.85 y un ROC-AUC de 0.99, resultados que se analizan en detalle en la sección de evaluación.

5.4 Análisis univariante

El análisis univariante tiene como objetivo estudiar la distribución individual de cada variable del dataset, con independencia de su relación con la variable objetivo. Este análisis permite identificar la naturaleza de cada atributo, detectar comportamientos atípicos y determinar qué transformaciones pueden ser necesarias antes del modelado.

5.4.1 Variables numéricas

La variable importe presenta una distribución fuertemente asimétrica hacia la derecha, con la mayoría de transacciones concentradas en valores bajos y una cola de operaciones de alto valor. Este comportamiento es coherente con el patrón habitual de consumo financiero, donde las compras cotidianas de pequeño importe son muy frecuentes y las de gran importe son puntuales. No se aplica transformación logarítmica dado que el modelo XGBoost es robusto frente a distribuciones asimétricas, pero la variable se escala mediante StandardScaler junto al resto de variables numéricas dentro del pipeline de preprocesamiento.

La variable hora toma valores enteros entre 0 y 23 y refleja el momento del día en que se produce la transacción. Su distribución muestra una concentración de operaciones durante el horario comercial, con un valle pronunciado en las horas de madrugada. Este patrón temporal resulta especialmente relevante para la detección de fraude, ya que las transacciones nocturnas presentan tasas de fraude superiores a la media, lo que motiva la creación de la variable derivada hora_madrugada en la fase de ingeniería de variables.

La variable dias_antigüedad_cuenta presenta una distribución aproximadamente uniforme entre cero y varios años, sin concentraciones destacables en ningún rango concreto.

Cuentas con muy pocos días de antigüedad pueden asociarse a un mayor riesgo, aunque la señal no es suficientemente fuerte de forma aislada como para ser determinante.

La variable `minutos_desde_ultima_tx` recoge el tiempo transcurrido desde la transacción anterior del mismo usuario. Presenta una distribución muy dispersa, con valores que van desde menos de un minuto hasta varios días, lo que refleja la variedad de patrones de uso entre distintos usuarios. Los valores muy bajos, correspondientes a transacciones muy seguidas en el tiempo, son un indicador potencial de card testing.

5.4.2 Variables categóricas

La variable `categoria` presenta seis valores únicos: Supermercado, Restaurante, Suscripciones, Salud, Viajes y Electrónica. Las categorías más frecuentes son Supermercado y Restaurante, que concentran la mayor parte de las transacciones. La categoría Electrónica, aunque minoritaria en volumen, presenta la tasa de fraude más elevada de todas las categorías (4.6%), lo que la convierte en una señal relevante para el modelo.

La variable `tipo_tarjeta` toma tres valores: Débito, Crédito y Prepago, con pesos aproximados del 60%, 35% y 5% respectivamente. Las tarjetas de débito y crédito presentan tasas de fraude similares (2.0%), mientras que las prepago muestran una tasa ligeramente inferior (1.8%), aunque su naturaleza no trazable las convierte en un factor de riesgo adicional que se captura mediante la variable codificada `tipo_tarjeta_Prepago` dentro del pipeline.

La variable `tipo_dispositivo` recoge cuatro valores: Móvil, Desktop, Tablet y Físico (TPV). Los dispositivos digitales (Móvil y Desktop) presentan tasas de fraude superiores (2.4%) frente a las transacciones presenciales mediante datáfono (1.4%), lo que es consistente con el mayor riesgo inherente a las operaciones online.

Las variables `pais_emision` y `pais_pago` presentan una distribución heterogénea entre múltiples países. España es el país de emisión y pago más frecuente, con diferencia. Desde el punto de vista del fraude, los países RU (10.3%), CN (8.4%) e IN (7.7%) concentran las tasas más elevadas en `pais_pago`, mientras que RU (6.4%), IN (5.1%) y CN (5.0%) lideran las tasas en `pais_emision`. Estos patrones motivan la creación de la variable derivada `pais_alto_riesgo` a partir de una lista de países con tasas de fraude significativamente superiores a la media, calculada sobre el conjunto de entrenamiento.

5.4.3 Variables binarias

Las variables binarias del dataset (`email_verificado`, `es_online`, `mismo_envio_facturacion`, `uso_vpn_proxy`, `paso_3d_secure`) presentan distribuciones muy desiguales entre sus dos categorías. El uso de VPN o proxy afecta a aproximadamente el 1% de las transacciones, y la ausencia de autenticación 3D Secure en operaciones online es relativamente poco frecuente. A pesar de su baja frecuencia absoluta, estas variables tienen una capacidad discriminativa elevada frente a la clase fraude y figuran entre las features más relevantes identificadas por el modelo, tal como confirma el análisis de importancia de variables mediante SHAP.

5.5 Análisis bivariate

El análisis bivariate estudia la relación entre cada variable del dataset y la variable objetivo `es_fraude`, con el fin de identificar qué atributos presentan mayor capacidad discriminativa entre transacciones legítimas y fraudulentas. Este análisis orienta tanto la selección de features como la interpretación posterior de los resultados del modelo.

Relación entre variables numéricas y fraude

La variable `importe` no presenta diferencias especialmente pronunciadas en su distribución media entre transacciones fraudulentas y legítimas cuando se analiza de forma aislada, lo

que sugiere que el importe por sí solo no es un predictor suficiente. Sin embargo, su interacción con otras variables —como el país de pago, el tipo de tarjeta o la ausencia de autenticación 3D Secure— incrementa notablemente su capacidad de señalización.

La variable hora muestra diferencias claras entre clases. Las transacciones fraudulentas presentan una sobrerrepresentación en las franjas horarias de madrugada (entre las 0 y las 5 horas), mientras que las transacciones legítimas se concentran mayoritariamente en horario diurno y de tarde. Esta diferencia motiva la creación de la variable derivada hora_madrugada, que captura de forma binaria esta señal y se convierte en la feature con mayor importancia en el modelo final según el análisis de feature importance de XGBoost, con un valor de 0.155.

La variable minutos_desde_ultima_tx presenta valores significativamente más bajos en las transacciones fraudulentas, lo que refleja que el fraude tiende a concentrarse en ráfagas de actividad intensa en periodos de tiempo muy cortos. Este patrón es especialmente característico del card testing, en el que se realizan múltiples transacciones de pequeño importe en muy poco tiempo para verificar la validez de una tarjeta robada.

La variable dias_antiguedad_cuenta muestra que las cuentas más recientes presentan tasas de fraude ligeramente superiores, aunque la diferencia no es suficientemente pronunciada como para convertirla en un predictor fuerte de forma aislada. Su valor discriminativo aumenta cuando se combina con otras señales de riesgo.

Relación entre variables categóricas y fraude

El análisis de las tasas de fraude por categoría revela diferencias significativas entre grupos. La categoría Electrónica presenta la tasa más elevada (4.6%), más del doble que la media del dataset. Le siguen Viajes (2.0%), Restaurante (1.7%), Supermercado (1.7%), Salud (1.7%) y Suscripciones (1.6%). Estas diferencias sugieren que la naturaleza del comercio donde se realiza la transacción contiene información relevante para la detección de fraude, y motivan la creación de la variable derivada categoria_alto_riesgo.

La variable pais_pago muestra las diferencias más pronunciadas de todo el dataset en términos de tasa de fraude por categoría. Los países RU (10.3%), CN (8.4%) e IN (7.7%) presentan tasas muy superiores a la media, mientras que ES (0.9%) se sitúa en el extremo opuesto como el país de pago más seguro. Esta disparidad justifica la construcción de la variable pais_alto_riesgo, derivada a partir de los países con tasas de fraude estadísticamente elevadas calculadas sobre el conjunto de entrenamiento, evitando así la fuga de información hacia los conjuntos de validación y test.

La variable tipo_dispositivo muestra que las transacciones presenciales mediante datáfono (Físico TPV) presentan la tasa de fraude más baja (1.4%), mientras que los dispositivos digitales se sitúan entre el 2.2% y el 2.4%. Esta diferencia es coherente con el mayor nivel de control físico que implica una operación presencial frente a una online.

Relación entre variables binarias y fraude

Las variables binarias presentan algunas de las relaciones más claras con la variable objetivo. La combinación de es_online = 1 y paso_3d_secure = 0 —capturada mediante la variable derivada online_sin_3ds— es una de las señales más potentes del dataset, con una importancia de 0.110 según el análisis de XGBoost. Las transacciones online que no superan la autenticación 3D Secure presentan tasas de fraude significativamente superiores a la media.

La variable mismo_envio_facturacion muestra una relación inversa con el fraude: las transacciones en las que la dirección de envío no coincide con la de facturación presentan mayor riesgo. Con una importancia de 0.140 en el modelo final, es la segunda feature más relevante tras hora_madrugada, lo que confirma que la discordancia entre ambas direcciones es un indicador robusto de comportamiento anómalo.

La variable `uso_vpn_proxy`, a pesar de afectar a un porcentaje muy reducido de transacciones, presenta una tasa de fraude notablemente superior entre los registros positivos, lo que confirma su utilidad como señal discriminativa dentro del pipeline.

Correlaciones entre variables numéricas

El análisis de correlaciones entre variables numéricas no revela relaciones lineales fuertes entre predictores, lo que descarta problemas de multicolinealidad graves que pudieran comprometer la estabilidad del modelo. Las correlaciones más relevantes se producen entre variables derivadas construidas a partir de las mismas fuentes originales, como `hora` y `hora_madrugada`, cuya relación es por construcción y no introduce información redundante dado que ambas capturan aspectos distintos del patrón temporal.

5.6 Análisis temporal

El dataset incluye información temporal asociada a cada transacción a través de la variable `fecha`, que registra el momento exacto en que se produce la operación con precisión de minutos. A partir de esta variable se extraen distintas dimensiones temporales que permiten estudiar si el fraude presenta patrones asociados a la hora del día, el día de la semana, el mes o la frecuencia de actividad del usuario.

Evolución del fraude en el tiempo

El dataset cubre un periodo temporal amplio que permite observar la distribución de transacciones y de casos de fraude a lo largo del tiempo. La tasa de fraude se mantiene aproximadamente estable a lo largo del periodo analizado, sin tendencias crecientes ni decrecientes pronunciadas que sugieran un cambio estructural en el patrón de fraude durante el horizonte de los datos. Esta estabilidad es consistente con la naturaleza sintética del dataset, donde la proporción de fraude se fija por construcción alrededor del 2%.

Estacionalidad por hora del día

La dimensión temporal más informativa es la hora de la transacción. El análisis de la tasa de fraude por franja horaria revela una sobrerrepresentación clara del fraude en las horas de madrugada, concretamente entre las 0 y las 5 horas, donde la tasa de fraude es significativamente superior a la media del dataset. Durante el horario diurno y de tarde, cuando el volumen de transacciones legítimas es mayor, la tasa de fraude desciende y se aproxima a la media global.

Este patrón tiene una explicación lógica desde el punto de vista del comportamiento fraudulento: las horas de madrugada presentan menor supervisión por parte de los usuarios y de los sistemas de alerta, lo que las convierte en una ventana de oportunidad para operaciones no autorizadas. En consecuencia, la variable derivada `hora_madrugada` —que toma el valor 1 para transacciones realizadas entre las 0 y las 5 horas y 0 en caso contrario— se convierte en la feature con mayor importancia en el modelo final, con un valor de 0.155 según el análisis de feature importance de XGBoost.

Estacionalidad por día de la semana

El análisis por día de la semana revela una ligera sobrerrepresentación del fraude durante los fines de semana, aunque la diferencia respecto a los días laborables no es tan pronunciada como la observada en la dimensión horaria. Este patrón motiva la creación de la variable derivada `es_fin_de_semana`, que captura de forma binaria si la transacción se produce en sábado o domingo, añadiendo una señal temporal adicional al conjunto de features del modelo.

Estacionalidad por mes

El análisis mensual no revela patrones de estacionalidad especialmente marcados en la tasa de fraude. La distribución de casos positivos a lo largo de los meses del año es aproximadamente uniforme, sin picos destacables asociados a periodos concretos como rebajas, campañas navideñas o fechas señaladas. Esta ausencia de estacionalidad

mensual es coherente con el carácter sintético de los datos y con la forma en que fueron generados.

Frecuencia de actividad y comportamiento temporal del usuario

Más allá de las dimensiones calendáricas, el análisis temporal incorpora también la variable `minutos_desde_ultima_tx`, que mide el tiempo transcurrido desde la transacción anterior del mismo usuario. Esta variable captura el ritmo de actividad individual y permite detectar ráfagas de transacciones en periodos muy cortos, un patrón especialmente asociado al card testing.

El análisis de esta variable segmentado por clase revela que las transacciones fraudulentas presentan valores de `minutos_desde_ultima_tx` significativamente más bajos que las legítimas, confirmando que el fraude tiende a concentrarse en ventanas temporales muy cortas. Este comportamiento refuerza la importancia de incorporar variables de frecuencia temporal en el pipeline de features, no solo como predictores individuales sino como señales que cobran mayor relevancia cuando se combinan con otras variables de riesgo como el importe, el país de pago o el uso de VPN.

Variables temporales derivadas

A partir de la variable fecha se construyen las siguientes variables derivadas, que sustituyen a la fecha original en el conjunto de features del modelo:

- **hora** — hora del día de la transacción, valor entero entre 0 y 23
- **día_semana** — día de la semana, valor entero entre 0 (lunes) y 6 (domingo)
- **es_fin_de_semana** — variable binaria, 1 si la transacción ocurre en sábado o domingo
- **mes** — mes del año de la transacción, valor entero entre 1 y 12
- **hora_madrugada** — variable binaria, 1 si la transacción ocurre entre las 0 y las 5 horas

Una vez extraídas estas variables, la columna fecha original se elimina del conjunto de features para evitar que el modelo aprenda patrones asociados a fechas concretas que no se generalizarán a datos futuros.

5.7 Detección de patrones de fraude

El análisis exploratorio realizado en las secciones anteriores permite identificar un conjunto de patrones de comportamiento consistentemente asociados a las transacciones fraudulentas. Estos patrones no actúan de forma aislada sino que se refuerzan mutuamente: la combinación de varias señales simultáneas incrementa de forma significativa la probabilidad de fraude, mientras que la presencia de una única señal puede corresponder a un comportamiento legítimo con una explicación razonable.

Variables más asociadas al fraude

El análisis de importancia de variables del modelo XGBoost, complementado con los valores SHAP calculados a nivel de transacción individual, identifica las siguientes variables como las más determinantes para la detección de fraude:

- `hora_madrugada` es la variable con mayor importancia global (0.155). Las transacciones realizadas entre las 0 y las 5 horas presentan una tasa de fraude notablemente superior a la media, lo que convierte esta franja horaria en uno de los indicadores más robustos del dataset.
- `mismo_envio_facturacion` ocupa el segundo lugar en importancia (0.140). La discordancia entre la dirección de envío y la de facturación es una señal fuerte de comportamiento anómalo, especialmente en compras online donde el defraudador utiliza una tarjeta ajena pero indica una dirección de entrega propia.

- `online_sin_3ds` es la tercera variable más relevante (0.110). La combinación de una transacción online con ausencia de autenticación 3D Secure elimina una barrera de seguridad clave y es un patrón recurrente en operaciones fraudulentas.
- `tipo_tarjeta_Prepago` (0.060) y `categoria_Electronica` (0.052) completan el grupo de variables con mayor capacidad discriminativa individual.

Segmentos con mayor riesgo

El análisis bivalente permite identificar los segmentos del dataset con mayor concentración de fraude. Las transacciones que combinan las siguientes características presentan tasas de fraude muy superiores a la media global del 2%: país de pago en la lista de países de alto riesgo (RU, CN, IN encabezan la lista con tasas del 10.3%, 8.4% y 7.7% respectivamente), transacción online sin autenticación 3D Secure, realizada en horario de madrugada, con tarjeta prepago o en la categoría Electrónica, y con discordancia entre dirección de envío y facturación. La presencia simultánea de tres o más de estas condiciones constituye el perfil de mayor riesgo identificado en el dataset.

Patrones por tipo de fraude

El análisis de los casos fraudulentos permite identificar subpatrones asociados a distintas modalidades de fraude:

- El robo de cuenta se caracteriza por la aparición de un dispositivo no habitual para el usuario, combinado con uso de VPN o proxy, país de pago distinto al habitual y operación en horario de madrugada.
- La tarjeta robada presenta un patrón de fallo en la autenticación 3D Secure junto a importes elevados y país de pago distinto al de emisión, frecuentemente con varios intentos de pago fallidos previos.
- El card testing se manifiesta como una ráfaga de transacciones de pequeño importe en un intervalo de tiempo muy corto, con valores de `minutos_desde_ultima_tx` muy bajos y presencia de intentos fallidos.
- El fraude internacional combina país de emisión distinto al país de pago, uso de VPN y dispositivo no habitual, en ocasiones con país de pago en la lista de alto riesgo.

Hipótesis derivadas del análisis

El análisis exploratorio genera las siguientes hipótesis que el modelo de machine learning se encarga de contrastar y cuantificar: las transacciones fraudulentas tienden a producirse en franjas horarias de menor actividad legítima, especialmente la madrugada. La ausencia de autenticación 3D Secure en operaciones online es un predictor robusto de fraude independientemente del resto de características de la transacción. El país de pago es más informativo que el país de emisión como señal de riesgo. La combinación de señales débiles es el mecanismo más habitual mediante el que se manifiesta el fraude en este dataset, lo que justifica el uso de un modelo de ensemble capaz de capturar interacciones entre variables.

5.8 Ingeniería de variables

La ingeniería de variables constituye una de las fases más relevantes del proceso analítico, ya que permite transformar las variables originales del dataset en representaciones más informativas para el modelo. En este proyecto, la creación de nuevas features responde directamente a los patrones identificados durante el análisis exploratorio y se implementa dentro del pipeline de preprocesamiento mediante una función reutilizable denominada `feature_engineering`, que se aplica de forma consistente tanto durante el entrenamiento como durante la inferencia en producción.

Criterios de diseño

El diseño de las variables derivadas sigue tres criterios principales. En primer lugar, cada nueva variable debe tener una justificación basada en el análisis exploratorio o en el

conocimiento del dominio del fraude financiero. En segundo lugar, las listas de referencia utilizadas para construir variables de riesgo se calculan exclusivamente sobre el conjunto de entrenamiento, evitando la fuga de información hacia los conjuntos de validación y test. En tercer lugar, la función de ingeniería de variables incorpora un mecanismo de idempotencia que impide que las transformaciones se apliquen dos veces sobre el mismo DataFrame.

Variables derivadas temporales

- hora — recoge la hora del día de la transacción como valor entero entre 0 y 23.
- dia_semana — codifica el día de la semana como valor entero entre 0 (lunes) y 6 (domingo).
- es_fin_de_semana — variable binaria, 1 cuando la transacción ocurre en sábado o domingo.
- mes — recoge el mes del año como valor entero entre 1 y 12.
- hora_madrugada — variable binaria, 1 para transacciones realizadas entre las 0 y las 5 horas. Es la feature con mayor importancia en el modelo final (0.155).

Variables derivadas geográficas

- pais_alto_riesgo — binaria, 1 cuando el país de pago pertenece a la lista de países con tasas de fraude superiores a la media, calculada sobre el conjunto de entrenamiento.
- pais_distinto — binaria, 1 cuando el país de emisión de la tarjeta difiere del país de pago.

Variables derivadas de comportamiento

- categoria_alto_riesgo — binaria, 1 cuando la categoría pertenece a la lista de categorías con tasas de fraude superiores a la media. La categoría Electrónica es la principal componente de esta lista.
- online_sin_3ds — binaria, 1 cuando la transacción es online (`es_online = 1`) y no ha superado la autenticación 3D Secure (`paso_3d_secure = 0`). Importancia en el modelo: 0.110.

Preprocesamiento dentro del pipeline

- Las variables numéricas se escalan mediante `StandardScaler`.
- Las variables categóricas se codifican mediante `OneHotEncoder` con `handle_unknown='ignore'`.
- Los valores nulos en `tipo_ip` se imputan con la categoría `Sin_IP` como paso previo al pipeline.

Separación de conjuntos

El dataset se divide en conjunto de entrenamiento y conjunto de test siguiendo una proporción 80/20, con estratificación sobre la variable objetivo. La técnica SMOTE se aplica exclusivamente sobre el conjunto de entrenamiento, después de la división, para evitar que las muestras sintéticas generadas contaminen la evaluación del modelo sobre datos no vistos.

5.9 Modelado

Modelos probados

Durante la fase de experimentación se evaluaron dos algoritmos de clasificación basados en árboles de decisión potenciados por gradient boosting: Random Forest y XGBoost. Ambos modelos son especialmente adecuados para problemas de clasificación con variables heterogéneas, desbalance de clases y necesidad de capturar interacciones no lineales entre predictores, características todas ellas presentes en este dataset.

Random Forest fue explorado como modelo de referencia por su robustez frente al sobreajuste y su capacidad para manejar variables categóricas y numéricas sin requerir transformaciones previas complejas. XGBoost fue seleccionado como modelo principal por su mayor capacidad para optimizar directamente métricas orientadas a la clase minoritaria, su eficiencia computacional y su mejor rendimiento empírico en el contexto del problema.

Justificación de la elección

La elección de XGBoost como modelo definitivo responde a varios criterios. En primer lugar, los modelos de gradient boosting son consistentemente competitivos en problemas de detección de fraude con datos tabulares, donde la captura de interacciones entre variables es determinante. En segundo lugar, XGBoost permite ajustar directamente el parámetro `scale_pos_weight` para compensar el desbalance de clases, aunque en este proyecto se optó por combinar SMOTE con hiperparámetros ajustados manualmente para maximizar el recall. En tercer lugar, la integración de XGBoost con SHAP permite obtener explicaciones a nivel de transacción individual.

Pipeline de entrenamiento

El pipeline integra de forma secuencial: función de ingeniería de variables (FunctionTransformer), un ColumnTransformer que aplica StandardScaler sobre numéricas y OneHotEncoder sobre categóricas, y el clasificador XGBoost. El pipeline completo se serializa como `modelo_fraude_v1.pkl`.

Ajuste de hiperparámetros

Los hiperparámetros del clasificador XGBoost se ajustaron de forma manual. Los valores finales adoptados son los siguientes:

```
XGBClassifier(  
    n_estimators      = 300,    # número de árboles del ensemble  
    max_depth         = 4,      # profundidad máxima de cada árbol  
    learning_rate      = 0.05,  # tasa de aprendizaje  
    subsample         = 0.8,    # fracción de muestras por árbol  
    colsample_bytree   = 0.8,    # fracción de features por árbol  
    random_state       = 42  
)
```

Manejo del desbalanceo

El desbalanceo entre clases se aborda mediante la aplicación de SMOTE exclusivamente sobre el conjunto de entrenamiento, con el objetivo de aumentar la representación de la clase minoritaria antes del ajuste del modelo. Esta técnica genera muestras sintéticas de transacciones fraudulentas interpolando entre ejemplos reales existentes. La aplicación de SMOTE se realiza después de la división train/test para garantizar que las muestras sintéticas no contaminen la evaluación del modelo sobre datos reales.

Umbral de decisión

Se analizó la curva precision-recall a distintos umbrales para identificar el punto de operación que maximiza el recall sin degradar la precisión por debajo de un nivel aceptable para el negocio. El umbral final se almacena junto al pipeline serializado dentro del artefacto `modelo_fraude_v1.pkl`.

A partir del análisis de la curva precision-recall, se establecen tres rangos de decisión operativa que el sistema aplica en producción. Las transacciones con una probabilidad de fraude inferior al 30% se clasifican automáticamente como legítimas y se aprueban sin intervención humana. Las que obtienen una probabilidad entre el 30% y el 50% se derivan a revisión manual por parte del analista, que dispone de las razones SHAP asociadas a cada caso para fundamentar su decisión. Las transacciones con una probabilidad superior al 50% se clasifican directamente como fraude y activan las acciones de bloqueo o verificación correspondientes según el nivel de alerta del sistema de reglas de negocio.

5.10 Evaluación de resultados

Métricas del modelo final

La evaluación del modelo se realiza sobre el conjunto de test, que no ha participado en ninguna fase del entrenamiento ni en la aplicación de SMOTE, garantizando así que los resultados reflejan el rendimiento real del sistema ante datos no vistos.

Métrica	Valor
Precisión (fraude)	0.97
Recall (fraude)	0.87
F1-score (fraude)	0.89
ROC-AUC	0.99

El ROC-AUC de 0.99 indica una capacidad discriminativa excepcional del modelo: es capaz de separar transacciones fraudulentas de legítimas con una fiabilidad muy alta en términos de probabilidad, independientemente del umbral de decisión aplicado. Este resultado confirma que el pipeline de features y el clasificador XGBoost capturan correctamente los patrones de fraude presentes en el dataset.

El recall de 0.87 sobre la clase fraude significa que el modelo detecta correctamente el 87% de todas las transacciones fraudulentas reales. En términos de negocio, esto implica que de cada 100 fraudes que se producen, el sistema identifica 87 de forma automática, derivando los casos más ambiguos hacia la revisión del analista mediante el mecanismo de semáforo descrito en la sección de arquitectura.

La precisión de 0.97 indica que prácticamente la totalidad de las alertas generadas por el modelo corresponden a fraudes reales. Este resultado refleja un sistema con muy baja tasa de falsas alarmas, lo que reduce significativamente la carga de revisión manual para el analista y mejora la experiencia del usuario al minimizar los bloqueos innecesarios de transacciones legítimas.

El F1-score de 0.89 refleja un equilibrio sólido entre precisión y recall, y constituye el indicador más completo del rendimiento global del modelo. Un valor de 0.89 sitúa al sistema en un nivel de rendimiento muy alto para un problema de detección de fraude con desbalanceo extremo de clases.

6. Estructura de la base de datos

6.1 Modelo de Entidad Relación

Se crea un modelo de entidad relación en MIRO:

Figura 1: Modelo de entidad - relación Novapay

6.2 Modelo lógico de la base de datos y normalización

Se crean tres tablas principales: clientes, país y transacciones.

Tabla de Clientes

```
CREATE TABLE clientes (
  id_usuario          UUID PRIMARY KEY,
  nombre              VARCHAR(30) NOT NULL,
  apellido            VARCHAR(30) NOT NULL,
  dni                 VARCHAR(20) UNIQUE,
  email               VARCHAR(150) UNIQUE,
  email_verificado    INT,
  dias_antiguedad_cuenta INTEGER,
  pais_emision        VARCHAR(2) NOT NULL,
  paso_3d_secure      BOOLEAN,
  bloqueado           BOOLEAN
);
```

Tabla de Países (Catálogo de referencia)

```
CREATE TABLE pais (
  codigo_iso  VARCHAR(2)  PRIMARY KEY,
  nombre_pais VARCHAR(100) NOT NULL
);

INSERT INTO pais (codigo_iso, nombre_pais) VALUES
('RU','Rusia'), ('ES','España'), ('US','Estados Unidos'),
('DE','Alemania'), ('BR','Brasil'), ('IT','Italia'),
('FR','Francia'), ('CN','China'), ('IN','India'),
('MX','México'), ('GB','Reino Unido'), ('JP','Japón'), ('PT','Portugal');
```

Se limitan los valores permitidos para el país.

Tabla de Transacciones

```
CREATE TABLE transacciones (
  id_transaccion      UUID PRIMARY KEY,
  id_usuario          UUID NOT NULL,
  fecha               DATE NOT NULL,
  hora                TIME NOT NULL,
  dias_antiguedad_cuenta INTEGER,
  email_verificado    BOOLEAN,
  pais_emision        VARCHAR(2) NOT NULL,
  categoria           VARCHAR(100),
  importe             NUMERIC(12,2) NOT NULL,
  es_online           BOOLEAN,
  pais_pago           VARCHAR(2) NOT NULL,
  tipo_tarjeta        VARCHAR(50),
  mismo_envio_facturacion BOOLEAN,
```

```
    tipo_dispositivo      VARCHAR(50),
    uso_vpn_proxy         BOOLEAN,
    paso_3d_secure        BOOLEAN,
    minutos_desde_ultima_tx INTEGER,
    es_fraude              BOOLEAN DEFAULT FALSE,
    CONSTRAINT fk_transacciones_usuario
        FOREIGN KEY (id_usuario) REFERENCES clientes(id_usuario)
);
```

7. Arquitectura del proyecto

7.1 Descripción de flujo de trabajo y tecnologías utilizadas

El flujo completo de ingeniería de datos, desarrollo de plataforma y analítica de fraude, fue desplegado sobre contenedores Docker, con el objetivo de garantizar reproducibilidad, modularidad y facilidad de despliegue.

En primer lugar, se diseñó y levantó una base de datos PostgreSQL en contenedores Docker para centralizar el almacenamiento de la información transaccional y de clientes, mientras que PGAdmin se empleó como herramienta de administración, consulta y visualización de la base de datos, facilitando tanto la exploración como el control de la estructura relacional.

Sobre esta base se generaron clientes ficticios para poblar la tabla de usuarios y se incorporaron nuevas columnas funcionales, como el campo "revisado", que corresponde a la revisión de la transacción por un analista de la empresa y el estado de esta luego de la revisión.

Posteriormente, se implementó una arquitectura de ingesta basada en Kafka para insertar flujos de transacciones en tiempo casi real, aprovechando su modelo de publicación y lectura de eventos para desacoplar productores y consumidores dentro del sistema.

En paralelo, se desarrollaron las APIs necesarias mediante FastAPI, creando los endpoints requeridos para la integración con la parte Full Stack y para la comunicación entre servicios de la plataforma.

Además, se incorporó una capa de auditoría y control, por parte de Ciberseguridad, orientada a supervisar el comportamiento del sistema y reforzar los mecanismos de revisión de operaciones sospechosas, complementando así la lógica técnica con medidas de gobernanza interna.

En la fase analítica, se contempló el reentrenamiento del modelo y la incorporación de SHAP como técnica de interpretabilidad, con el propósito de explicar la contribución de las variables a las predicciones y mejorar la transparencia del sistema en la detección de fraude.

Finalmente, todos los servicios fueron dockerizados para simplificar su despliegue y mantenimiento, y se utilizó Cloudflare Tunnel como mecanismo adicional de seguridad, ya que permite exponer servicios sin necesidad de direcciones IP públicas ni puertos de entrada abiertos, mediante conexiones salientes seguras hacia la red de Cloudflare.

Figura 2: Arquitectura del proyecto

Resumen tecnológico

Componente	Tecnología
Framework	FastAPI (async) + Uvicorn
ORM	SQLModel / SQLAlchemy 2.0
Bases de datos	PostgreSQL (asyncpg) + Valkey/Redis (aioredis)
Autenticación	JWT (HS256) + bcrypt password hashing
Modelo ML	XGBoost pipeline (modelo_fraude_v1.pkl) + SHAP
Tablas BD	clientes, transacciones, analistas
Despliegue	Docker + docker-compose (port 8000)
Entry point	src/api/app.py — uvicorn app:app --host 0.0.0.0 --port 8000

Antes de que una transacción llegue al modelo de predicción, la API aplica una serie de filtros de validación que garantizan la integridad del sistema y previenen comportamientos maliciosos. En primer lugar, se comprueba que el identificador de transacción no exista ya en la base de datos, descartando peticiones duplicadas que pudieran distorsionar el historial. A continuación, se verifica que el usuario que realiza la solicitud exista en la base de datos de clientes. Si el usuario está bloqueado por haber superado el límite de intentos permitidos, la petición es rechazada directamente. Finalmente, se controla la frecuencia de envío: si un mismo usuario realiza más de tres transacciones en una ventana de diez segundos, la solicitud no pasa el filtro. Solo las transacciones que superan todos estos controles acceden a la capa de inferencia del modelo.

7.2 Kafka

7.2.1 Generación de datos - Producer

Se crea un archivo `producer.py`, que genera transacciones ficticias y las publica en el topic transacciones de Kafka, simulando un flujo continuo de datos para probar el sistema de detección de fraude.

El script carga una lista de usuarios válidos desde `data/processed/id_users.csv`, y luego crea un `KafkaProducer` conectado al servidor definido en la variable de entorno `KAFKA_BOOTSTRAP` o, por defecto, a `localhost:9092`.

A partir de ahí entra en un bucle infinito en el que genera entre 1 y 5 transacciones por iteración, construye cada transacción con campos como: `id_transaccion`, `id_usuario`, `categoria`, `es_online`, `importe`, `minutos_desde_ultima_tx`, `mismo_envio_facturacion`, `pais_pago`, `tipo_dispositivo`, `tipo_tarjeta` y `uso_vpn_proxy`, y la publica en Kafka usando `producer.send('transacciones', tx)`.

Las transacciones son ficticias y siguen distribuciones que intentan parecer realistas. Entre los criterios utilizados: la mayoría de pagos se asignan a España, las categorías más frecuentes son supermercado y restaurante, hay más tarjetas de débito que de crédito o prepago, y el uso de VPN/proxy es muy raro. También se controla el patrón de compra con una aleatoriedad temporal entre envíos usando `time.sleep(random.expovariate(3))`, lo que hace que las publicaciones no sean constantes sino más parecidas a un flujo de eventos real. Se genera un máximo de 5% de transacciones fraudulentas.

7.2.2 Consumer

Se crea el archivo `consumer.py`, el cual funciona como un consumer de Kafka que lee transacciones del tópico `transacciones` y las envía a una API de predicción de fraude para obtener una clasificación.

El script crea un `AIOKafkaConsumer` conectado a Kafka mediante `KAFKA_BOOTSTRAP` o, si no está definida esa variable, a `localhost:9092`. Se suscribe al tópico `transacciones`, usa el grupo `prediccion-group` y deserializa cada mensaje desde JSON. Luego comienza a leer eventos desde el inicio si no hay un offset previo guardado.

El flujo de trabajo es el siguiente: cuando llega una transacción, el consumer la toma desde Kafka y la envía por POST a la URL del proyecto usando un cliente HTTP asíncrono. Si la API responde correctamente con código 200, el consumer confirma el mensaje en Kafka con `commit()` para no procesarlo otra vez, extrae la predicción y la probabilidad de fraude, y muestra por pantalla si la transacción es fraude o legítima junto con el usuario y la probabilidad.

Si la API devuelve un error o ocurre una excepción, el script espera 2 segundos y vuelve a intentar, lo que ayuda a mantener el proceso vivo ante fallos temporales. Además, al tener `enable_auto_commit=False`, el control de confirmación del mensaje es manual, así que solo se marca como procesado cuando la predicción se recibe correctamente.

7.3 SHAP

SHAP asigna a cada feature una contribución individual a la predicción expresada como valor numérico. Los valores positivos indican que la variable empuja la predicción hacia fraude; los valores negativos indican que la reduce. Para cada transacción evaluada se extraen las cinco variables con mayor impacto absoluto, separadas en razones de fraude y razones de legitimidad.

El primer ejemplo corresponde a una transacción clasificada como fraude probable, donde las variables activas empujan la probabilidad hacia la clase positiva:

```
{
  "es_fraude": true,
  "probabilidad_fraude": 0.87,
  "razones_fraude": [
    {"feature": "hora_madrugada", "impacto": +0.812},
    {"feature": "online_sin_3ds", "impacto": +0.743},
    {"feature": "pais_alto_riesgo", "impacto": +0.601},
    {"feature": "uso_vpn_proxy", "impacto": +0.489},
    {"feature": "tipo_tarjeta_Prepago", "impacto": +0.312}
  ],
  "razones_legitima": []
}
```

El segundo ejemplo corresponde a una transacción clasificada como legítima, donde las variables activas reducen la probabilidad de fraude:

```
{
  "es_fraude": false,
  "probabilidad_fraude": 0.04,
  "razones_fraude": [],
  "razones_legitima": [
    {"feature": "online_sin_3ds", "impacto": -0.940},
    {"feature": "uso_vpn_proxy", "impacto": -0.853},
    {"feature": "pais_distinto", "impacto": -0.602},
    {"feature": "dias_antiguedad_cuenta", "impacto": -0.588},
  ]
}
```

```
    {"feature": "categoria_Restaurante",      "impacto": -0.555}  
  ]  
}
```

La interpretación del primer caso es directa: la transacción ocurre de madrugada, no supera la autenticación 3D Secure, el país de pago está en la lista de alto riesgo, se detecta uso de VPN y se realiza con tarjeta prepago. Ninguna de estas señales es suficiente por sí sola, pero su combinación eleva la probabilidad de fraude hasta el 87%. El segundo caso ilustra el comportamiento opuesto: la ausencia de las señales más determinantes reduce la probabilidad de fraude al 4%.

Esta información se almacena en el campo `shap_reasons` de la tabla de transacciones y se expone a través de la API para que el analista pueda consultar la justificación de cada predicción en el portal de revisión.

7.4 APACHE SUPERSET

Para facilitar la labor de supervisión y revisión de transacciones, se integra Apache Superset como herramienta de visualización conectada directamente a la base de datos PostgreSQL del sistema. Superset es una plataforma de inteligencia de negocio open source que permite explorar datos, construir dashboards interactivos y filtrar información, con la opción de no usar código en caso de no ser necesario.

La conexión se establece directamente sobre la base de datos PostgreSQL donde se persisten todas las transacciones evaluadas por el modelo, incluyendo el score de fraude, el nivel de alerta, las razones SHAP y el estado de revisión de cada operación. Esto permite al analista consultar el histórico completo de predicciones y filtrar por cualquier combinación de variables directamente desde el dashboard, sin necesidad de acceder a la API ni a la base de datos de forma manual.

7.5 PORTAL DEL ANALISTA

Es la interfaz operativa desarrollada por el equipo Full Stack para revisar transacciones con posible riesgo de fraude. Su objetivo es centralizar en una única herramienta el trabajo diario del analista: consultar operaciones pendientes, analizar su información asociada y registrar una decisión.

El portal incluye un dashboard inicial con indicadores generales, una vista de transacciones pendientes, una pantalla de detalle avanzado de transacción y una sección de clientes. Desde estas vistas, el analista puede revisar el contexto de cada operación, acceder al perfil del usuario relacionado y ejecutar acciones como aprobar una transacción, marcarla como fraude o bloquear un cliente.

La interfaz fue diseñada para ser clara, responsive y usable tanto en escritorio como en móvil, priorizando la lectura rápida de información crítica y la toma de decisiones. Además, se incorporaron confirmaciones visuales, estados de carga, paginación, filtros y animaciones suaves para mejorar la experiencia de uso.

Esta sección representa el punto de unión entre los modelos desarrollados por Data Science, las medidas de protección de Ciberseguridad y la capa de aplicación construida por Full Stack.

7.6 AUDITORÍA DE SEGURIDAD

7.6.1 Metodología

Aquí va:

7.6.1 Metodología

La vertical de ciberseguridad operó como Red Team, es decir, como el componente ofensivo del esquema adversarial. Su objetivo no fue defender el sistema sino someterlo a presión: diseñar y ejecutar fraude sintético contra el endpoint de scoring `/predict/` para medir qué consigue evadir la detección y por qué. El trabajo se organizó en dos rondas y se apoyó en un cliente de pruebas propio que encapsula la lógica de envío, la limitación de frecuencia y el registro estructurado de cada petición y respuesta.

Superficie de ataque. El primer trabajo técnico fue delimitar qué puede y qué no puede controlar un atacante sobre el payload de `/predict/`. El análisis distinguió tres grupos de variables: las directamente inyectables en el JSON de la petición (categoría, importe, país de pago, tipo de tarjeta, dispositivo, uso de VPN, etc.); las que el backend resuelve mediante JOIN con la tabla de clientes a partir del `id_usuario` (email_verificado, paso_3d_secure, dias_antigüedad_cuenta, pais_emision), que el atacante solo controla indirectamente eligiendo a qué usuario suplanta; y las que el servidor fija de forma autónoma, como la hora de la transacción, no inyectables desde el cliente. Esta segmentación condiciona todo el diseño de los ataques posteriores.

Controles previos al modelo. Se verificó que, antes de alcanzar la capa de inferencia, la API aplica una cadena de filtros de validación: rechazo de transacciones duplicadas, existencia del usuario, estado de bloqueo y control de frecuencia por usuario. Estos controles constituyen una defensa en profundidad correcta: actúan como barrera de negocio previa al modelo y rechazan peticiones sin coste de inferencia. Durante la segunda ronda, un 27% de las peticiones fue rechazado por este mecanismo al corresponder a usuarios del pool que ya se encontraban en estado bloqueado, lo que validó positivamente que el filtrado de entrada opera con anterioridad a la predicción.

Puntos ciegos identificados. El análisis adversarial reveló que el modelo evalúa cada transacción como un evento aislado, sin contexto histórico del usuario, lo que permite que un atacante con credenciales válidas opere con impunidad mientras mantenga importes moderados. Asimismo, el campo temporal que declara el intervalo entre transacciones se acepta tal y como lo envía el cliente, sin contraste con el tiempo real en servidor, lo que deja la velocidad de operación fuera del alcance efectivo del modelo. Finalmente, se confirmó que la geografía del pago es la señal individual más determinante del clasificador. Las métricas completas de evasión por tipología y su evolución entre rondas se recogen en los informes de Red Team.

Hallazgos de auditoría no explotados. Durante la revisión de seguridad de la API se identificó una vulnerabilidad de inyección de tipo NoSQL. Se tomó la decisión deliberada de no explotarla, dada la ausencia de copias de seguridad de la base de datos y la proximidad de la fecha de entrega: su explotación podría haber dejado el sistema inoperable para el resto de verticales. El hallazgo se documentó con su recomendación de mitigación. Por el mismo criterio, la cola de ingesta Apache Kafka se consideró un vector teórico adicional que queda fuera del alcance adversarial de este proyecto, centrado en la evasión del modelo y no en el pentesting de infraestructura.

7.6.2 Ronda 1: Hallazgos principales

Se evaluaron 170 transacciones sobre cuatro tipologías complementarias. Los resultados más relevantes fueron los siguientes.

La suplantación silenciosa (Account Takeover) evadió al 100%: 57 de 57 transacciones pasaron sin ser detectadas con un score medio del 0.91%. El modelo no dispone de features de comportamiento histórico por usuario, por lo que es estructuralmente incapaz de distinguir al titular real de un atacante que replica su comportamiento habitual.

El card testing también evadió al 100%: 39 de 39 transacciones con microtransacciones de entre 0.50€ y 8€ en ráfaga pasaron sin ser detectadas. La causa es que el campo `minutos_desde_ultima_tx` lo declara el cliente en el payload y el modelo lo acepta de forma pasiva, sin contrastar el tiempo real entre llamadas.

La ablación de features individuales confirmó que ninguna señal aislada —VPN, tarjeta prepago, envío distinto a facturación— es suficiente para disparar la detección por sí sola. El único factor con peso individual significativo fue `pais_pago = RU`, que activó un score del 71.9% en aislado.

Se identificó además un sesgo categórico explotable: importes hasta 30 veces la mediana habitual de la categoría Supermercado no superaron el 10% de score. El modelo trata ciertas categorías como zonas seguras sin condicionar por importe ni por el histórico del usuario.

En paralelo, el Red Team identificó una vulnerabilidad crítica de inyección NoSQL (Array Injection) en el endpoint `/trans/` que permitía el bypass total de autorización y la extracción masiva de transacciones de todos los usuarios. El hallazgo fue entregado al Blue Team para su mitigación sin ser explotado como vector adversarial, dada la proximidad de la fecha de entrega.

7.6.3 Ronda 2: Evaluación modelo reentrenado

Se evaluaron 85 sondas sobre el endpoint `/predict/` con la versión reentrenada del modelo. El 27% de las peticiones fueron rechazadas con HTTP 403 antes de llegar al modelo, lo que validó positivamente que el filtro de bloqueo actúa correctamente en la capa de API.

Sobre las 62 transacciones que completaron la inferencia, los resultados mostraron mejoras significativas en algunas tipologías y persistencia de vulnerabilidades en otras. La combinación de país de alto riesgo con canal online fue bloqueada en el 86% de los casos, y el apilamiento de señales blandas (tarjeta prepago más envío distinto a facturación) produjo bloqueos con score medio del 81%.

Sin embargo, la suplantación silenciosa y el card testing mantuvieron una tasa de evasión del 100%, confirmando que ninguna de las dos recomendaciones críticas de Ronda 1 —features de comportamiento histórico y velocity server-side— había sido implementada. Se identificó además una costura explotable en la penalización por importe: categorías como Salud y Restaurante siguen absorbiendo importes anómalos sin activar la detección.

7.6.4 Recomendaciones pendientes

El Red Team documentó las siguientes líneas de mejora ordenadas por impacto, que quedan registradas como trabajo futuro del sistema:

- Implementar features de comportamiento histórico por usuario (z-score de importe por categoría, novedad de país y dispositivo, horario típico), lo que cerraría la vulnerabilidad ante suplantación mimética.
- Calcular la velocity de transacciones en el servidor a partir de marcas de tiempo reales, sustituyendo el campo declarado en el payload.
- Introducir el importe relativo al historial del usuario (z-score condicional por usuario y categoría) en lugar del importe absoluto, eliminando las zonas seguras por categoría.

8. Sistema de reglas de negocio

8.1 Descripción general

El sistema de reglas de negocio constituye una capa de decisión complementaria al modelo de machine learning. Su función no es sustituir al modelo sino interpretar y contextualizar su output, traduciendo la probabilidad de fraude en una acción concreta y explicable para el analista. Mientras el modelo XGBoost produce un score continuo entre 0 y 1, el sistema de reglas convierte ese score en un nivel de alerta estructurado, asigna un tipo de fraude probable y recomienda una acción operativa.

El principio fundamental del sistema es que ninguna señal aislada activa una decisión de fraude. Es la combinación de señales y su gravedad conjunta lo que determina el nivel de alerta. Una transacción con VPN activa no es necesariamente fraude; una transacción con VPN, país distinto, dispositivo nuevo y operación de madrugada sí constituye un patrón de alto riesgo.

8.2 Flujo de decisión

El sistema ejecuta cuatro funciones en orden secuencial, integradas en la función principal `evaluar_transaccion()`:

```
def evaluar_transaccion(transaccion):
    senales = detectar_senales(transaccion)
    nivel_alerta = clasificar_alerta(senales)
    tipo_fraude = detectar_tipo_fraude(senales)
    accion = recomendar_accion(nivel_alerta)
    return {
        'id_transaccion': transaccion['id_transaccion'],
        'senales_detectadas': senales,
        'nivel_alerta': nivel_alerta,
        'tipo_fraude': tipo_fraude,
        'accion_recomendada': accion
    }
```

8.3 Detección de señales

La función `detectar_senales()` recibe los datos de la transacción y devuelve una lista de señales sospechosas activas. Las señales contempladas son las siguientes: `uso_vpn_proxy`, `pais_distinto`, `fallo_3d_secure`, `dispositivo_nuevo`, `importe_alto` y `transacciones_muy_seguidas`. Cada señal responde a una condición concreta sobre los datos de la transacción y se activa de forma independiente, sin implicar por sí sola ninguna decisión sobre la legitimidad de la operación.

8.4 Clasificación de alertas

La función `clasificar_alerta()` convierte el número de señales detectadas en un nivel de alerta siguiendo la siguiente escala:

Señales	Nivel	Decisión
0	Nivel 0	No fraude
1	Nivel 1	Alerta baja

Señales	Nivel	Decisión
2	Nivel 2	Alerta media
3	Nivel 3	Alerta alta
4 o más	Nivel 4	Fraude probable

El nivel 5 —fraude confirmado— únicamente es alcanzable mediante validación explícita del analista en el portal de revisión, cuando confirma que el usuario no reconoce la operación o cuando existe evidencia directa de uso no autorizado. En ese momento se registra `es_fraude = true` y se asigna el campo `tipo_fraude` en la base de datos.

8.5 Detección del tipo de fraude

La función `detectar_tipo_fraude()` analiza la combinación de señales activas y asigna el patrón de fraude más probable. Esta función no confirma el fraude sino que orienta al analista indicando qué tipo de comportamiento fraudulento podría estar produciéndose. Los tipos contemplados son los siguientes:

- `robo_cuenta` — se activa cuando coinciden dispositivo nuevo, VPN activa y país distinto, sugiriendo que un tercero ha tomado el control de las credenciales del usuario legítimo.
- `tarjeta_robada` — se activa cuando coinciden fallo de 3D Secure, importe alto y país distinto, patrón compatible con el uso de datos de tarjeta sustraída antes de que el titular la reporte.
- `card_testing` — se activa cuando hay transacciones muy seguidas sin importe alto, patrón característico de la verificación masiva de tarjetas robadas mediante micropagos.
- `fraude_internacional` — se activa cuando coinciden país distinto y VPN activa, sugiriendo un intento de operación desde una ubicación geográficamente alejada del perfil habitual del usuario.
- `compra_anomala` — se activa cuando el importe es significativamente superior al habitual del usuario, sin que otras señales permitan asignar un tipo más específico.
- `evasion_seguridad` — se activa cuando coinciden VPN activa y fallo de 3D Secure, patrón compatible con un intento deliberado de evitar los controles de autenticación del sistema.

8.6 Acción recomendada

La función `recomendar_accion()` traduce el nivel de alerta en una acción operativa concreta:

Nivel	Acción
Nivel 0	Aprobar transacción
Nivel 1	Aprobar y registrar alerta
Nivel 2	Marcar para seguimiento
Nivel 3	Enviar a revisión del analista
Nivel 4	Bloquear temporalmente o pedir verificación
Nivel 5	Registrar como fraude confirmado y asignar tipo

8.7 Relación con el modelo ML

El sistema de reglas de negocio y el modelo XGBoost operan de forma complementaria. El modelo proporciona un score de probabilidad continuo (0.0–1.0) con una capacidad discriminativa muy alta (ROC-AUC 0.99), pero no ofrece por sí solo una acción operativa ni una explicación del tipo de fraude detectado. El sistema de reglas aporta la capa de interpretabilidad y decisión que convierte esa probabilidad en una respuesta accionable para el analista.

Adicionalmente, el sistema de reglas actúa como salvaguarda ante casos que el modelo podría no capturar bien de forma aislada, como patrones de ráfaga temporal —que requieren comparar varias transacciones en secuencia— o situaciones donde el contexto del usuario reduce el riesgo real de una operación con score elevado. Esta combinación de modelo estadístico y reglas de negocio explícitas constituye una arquitectura de decisión más robusta y auditable que cualquiera de las dos capas por separado.

9. Conclusiones

El proyecto NovaPay ha permitido construir un sistema de detección de fraude end-to-end funcional, que va desde la generación y análisis de los datos hasta el despliegue de una API en producción con capacidad de procesar transacciones en tiempo real. El resultado no es únicamente un modelo predictivo, sino una plataforma integrada en la que cada componente cumple una función específica dentro de un flujo coherente y auditable.

Sobre el modelo y los datos

El modelo XGBoost entrenado sobre el dataset sintético de 100.000 transacciones obtiene resultados sólidos en las métricas más relevantes para el contexto del fraude: un ROC-AUC de 0.99 confirma una capacidad discriminativa excepcional, y un recall de 0.79 sobre la clase fraudulenta significa que el sistema detecta casi ocho de cada diez fraudes de forma automática. Estos resultados son especialmente significativos considerando el desbalanceo extremo del dataset, donde la clase fraudulenta representa únicamente el 2% de las observaciones.

Una de las decisiones más relevantes del proceso fue priorizar el recall frente a la precisión como métrica guía. Esta elección no es arbitraria sino que responde a una comprensión del coste asimétrico de los errores en el negocio del fraude: dejar pasar un fraude tiene consecuencias económicas directas e irreversibles, mientras que bloquear una transacción legítima genera fricción pero es recuperable mediante la revisión del analista.

El análisis exploratorio reveló que las señales más potentes no son las más obvias. Variables como hora_madrugada, mismo_envio_facturacion y online_sin_3ds —ninguna de las cuales representa por sí sola un indicador definitivo de fraude— se convierten en los predictores más determinantes cuando el modelo aprende a combinarlas. Esta lección subraya la importancia de dedicar tiempo al análisis de los datos antes de modelar: las mejores features no siempre están en el dataset original sino que emergen del conocimiento del dominio aplicado sobre los datos brutos.

Sobre la arquitectura y el despliegue

La decisión de construir el sistema sobre una arquitectura de microservicios dockerizados, con PostgreSQL para la persistencia, Valkey para el enriquecimiento en tiempo real y Kafka para la ingesta de eventos, responde a criterios de escalabilidad y separación de responsabilidades que van más allá de los requisitos mínimos del proyecto. El resultado es un sistema que puede procesar flujos continuos de transacciones, enriquecer cada operación con el perfil histórico del usuario en milisegundos, registrar el historial completo de predicciones y exponer toda esa información a través de una API segura con autenticación JWT.

La incorporación de SHAP como capa de interpretabilidad es quizás uno de los elementos más valiosos del sistema desde el punto de vista operativo. Convertir cada predicción en una lista de razones comprensibles para el analista transforma el modelo de una caja negra en una herramienta auditable, lo que resulta imprescindible en un entorno financiero regulado donde cada decisión debe poder justificarse.

Sobre el proceso y el trabajo en equipo

El proyecto ha requerido coordinar tres verticales con tecnologías y objetivos distintos —análisis de datos y machine learning, desarrollo de plataforma e infraestructura, y ciberseguridad— en torno a un producto común. Esta coordinación no ha sido trivial: las decisiones tomadas en una vertical afectan directamente al trabajo de las otras, y alinear los esquemas de datos, los contratos de la API y los flujos de ingesta ha exigido comunicación constante y documentación clara de las interfaces entre componentes.

El resultado demuestra que es posible construir un sistema de detección de fraude completo y funcional en un tiempo acotado cuando las decisiones técnicas están bien justificadas y cada componente tiene una responsabilidad clara dentro del sistema. No todo ha quedado

perfecto —hay elementos que con más tiempo se refinarían, como la calibración del umbral de decisión, la cobertura de los endpoints del portal del analista o la implementación del agente de IA contemplado como extensión futura— pero el núcleo del sistema es sólido, coherente y defendible.

10. Herramientas y tecnologías

Lo he leído. Mi recomendación es **no separar por vertical** — queda más profesional y evita la sensación de que son dos proyectos distintos pegados. Lo organizo por área funcional como está ahora, simplemente integrando lo de Full Stack donde corresponde.

Hay algunas repeticiones a resolver: JWT y bcrypt aparecen en ambos textos, PostgreSQL también, y Docker/Vercel/Render son capas de despliegue que van juntas. Lo fusiono todo:

10. Herramientas y tecnologías

El desarrollo del proyecto ha requerido el uso de un conjunto amplio de herramientas y tecnologías, organizadas por área funcional.

Lenguaje de programación

El backend de datos y la API principal están desarrollados en Python 3.11. El portal del analista utiliza Node.js con Express en el backend y React con Vite en el frontend.

Análisis y modelado

Para el análisis exploratorio y la preparación de datos se utilizaron pandas y numpy. La visualización de distribuciones, tasas de fraude y feature importance se realizó con matplotlib y seaborn. El modelo de machine learning se construyó con xgboost y el pipeline de preprocesamiento con scikit-learn, incluyendo StandardScaler, OneHotEncoder y ColumnTransformer. El manejo del desbalanceo de clases se realizó mediante imbalanced-learn y su implementación de SMOTE. La interpretabilidad del modelo se implementó con shap, utilizando TreeExplainer para la generación de valores SHAP a nivel de transacción individual. La serialización del pipeline completo se realizó con joblib.

Generación de datos sintéticos

Los perfiles de usuarios ficticios se generaron con Faker, utilizando configuraciones regionales múltiples para simular una base de clientes con nombres españoles e internacionales. Los identificadores únicos de usuario y transacción se generaron como UUID mediante el módulo estándar uuid de Python.

API y backend

La API principal de predicción se desarrolló con FastAPI en modo asíncrono, servida mediante Uvicorn. El ORM utilizado es SQLAlchemy sobre SQLAlchemy 2.0 con soporte asíncrono a través de asyncpg. La validación de esquemas de entrada y salida se gestiona con Pydantic. El portal del analista dispone de su propio backend desarrollado con Node.js y Express, que se comunica con la API principal mediante Axios. La autenticación en ambas capas se implementa mediante tokens JWT y hashing de contraseñas con bcrypt, con medidas adicionales de seguridad mediante Helmet y CORS en el portal. La navegación del frontend se gestiona con React Router, los estilos con CSS Modules, y se incorporaron Lucide React para iconografía, Framer Motion para animaciones y SweetAlert2 para confirmaciones visuales.

Bases de datos

La persistencia principal del sistema se apoya en PostgreSQL, administrada durante el desarrollo con pgAdmin. La gestión de usuarios analistas del portal se realiza sobre una instancia PostgreSQL desplegada en Supabase, accedida desde el backend del portal con Sequelize. Para el almacenamiento en memoria de perfiles de usuario y el control de intentos en tiempo real se utiliza Valkey, compatible con el protocolo Redis y accedido desde Python mediante redis en modo síncrono y aioredis en modo asíncrono.

Ingesta de eventos

La arquitectura de ingesta en tiempo real se implementó con Apache Kafka, utilizando kafka-python para el producer síncrono y aiokafka para el consumer asíncrono. El consumer se comunica con la API mediante el cliente HTTP asíncrono httpx.

Visualización y análisis operativo

Para la exploración del histórico de transacciones y la supervisión del sistema en producción se integró Apache Superset, conectado directamente a la base de datos PostgreSQL.

Despliegue e infraestructura

La API principal y sus servicios asociados se desplegaron sobre contenedores Docker orquestados mediante docker-compose. La exposición segura del servicio sin necesidad de puertos de entrada abiertos se implementó con Cloudflare Tunnel. El frontend del portal del analista se desplegó en Vercel y su backend en Render, manteniendo la comunicación entre ambos mediante variables de entorno y conexión segura con Supabase. El entorno de desarrollo utilizó Visual Studio Code con extensiones de Python y Docker.

Control de versiones

El código del proyecto se gestiona mediante Git con repositorio privado en GitHub, siguiendo una estructura de ramas por vertical para facilitar la integración entre equipos.

Documentación y diseño

El modelo de entidad-relación de la base de datos se diseñó en Miro. La documentación interactiva de la API se genera automáticamente mediante Swagger UI, integrado en FastAPI y accesible en producción en <https://business.1morecoffee.cc/docs>.

11. Líneas futuras de proyecto

El sistema desarrollado constituye una solución funcional y coherente, pero como todo proyecto acotado en tiempo y recursos, presenta áreas de mejora que conviene revisar en el futuro.

Líneas futuras

- Crear una tabla de analista independiente de la tabla de transacciones, que centralice la información de seguimiento al cliente y el control de fraude de forma separada, e implementar un sistema de filtrado y asignación de transacciones según el monto o la especialización y experiencia de cada analista.
- Implementar un pipeline de reentrenamiento periódico. A medida que el sistema acumule transacciones revisadas con etiquetas confirmadas, ese histórico puede utilizarse para reentrenar el modelo de forma regular, incorporando los patrones de fraude más recientes y reduciendo el riesgo de degradación del rendimiento con el tiempo.
- Incorporar un canal de comunicación automática entre NovaPay y el cliente a través de la app bancaria, que permita notificar alertas en tiempo real, solicitar verificación directa al usuario y registrar su respuesta como señal adicional para el sistema de decisión.
- En términos de normativa y cumplimiento, implementar mecanismos de anonimización y seudonimización de los datos personales almacenados, la definición de políticas de retención y el desarrollo de un registro de auditoría de accesos que facilite el cumplimiento del RGPD en un entorno de producción real.
- Revisión historial de los clientes